



VISION AND LEARNING LAB

MÔ HÌNH PHÂN LOẠI

TS. Hoàng Quốc Việt



HUNG YEN UNIVERSITY OF TECHNOLOGY AND EDUCATION



Giới thiệu



Phân loại nhị phân

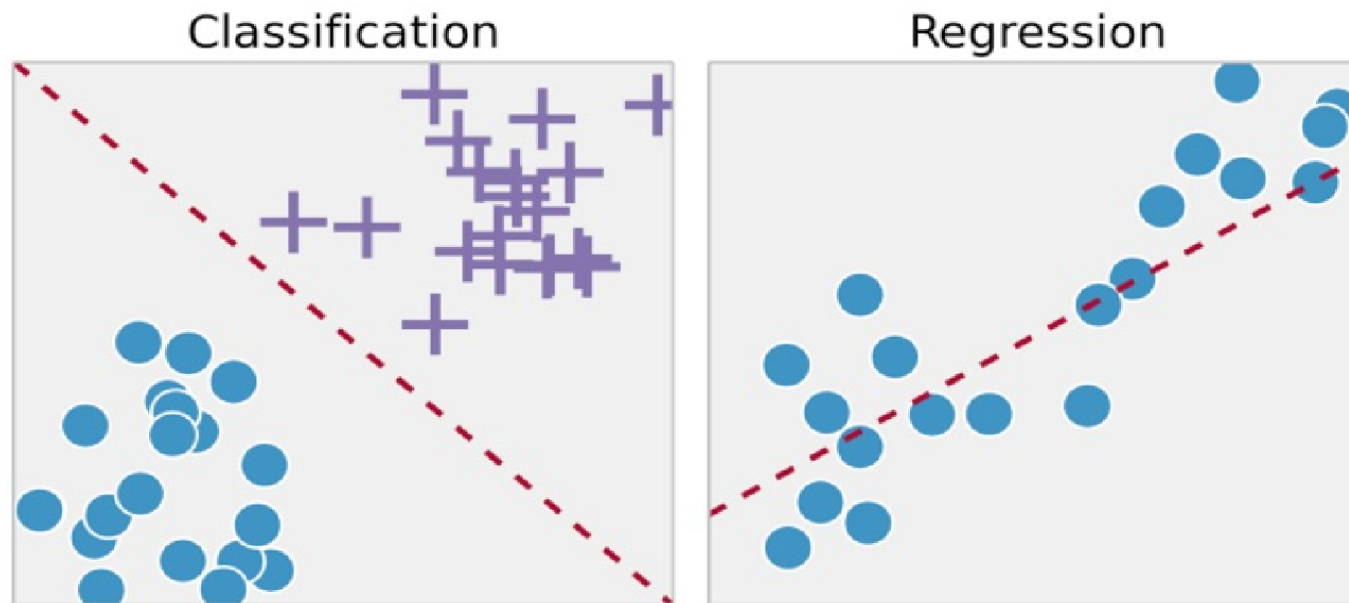


Phân loại đa lớp



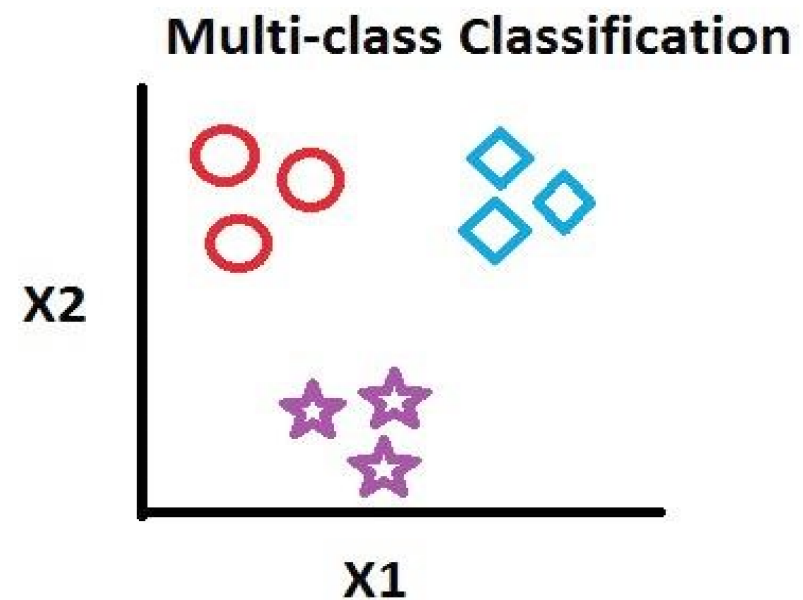
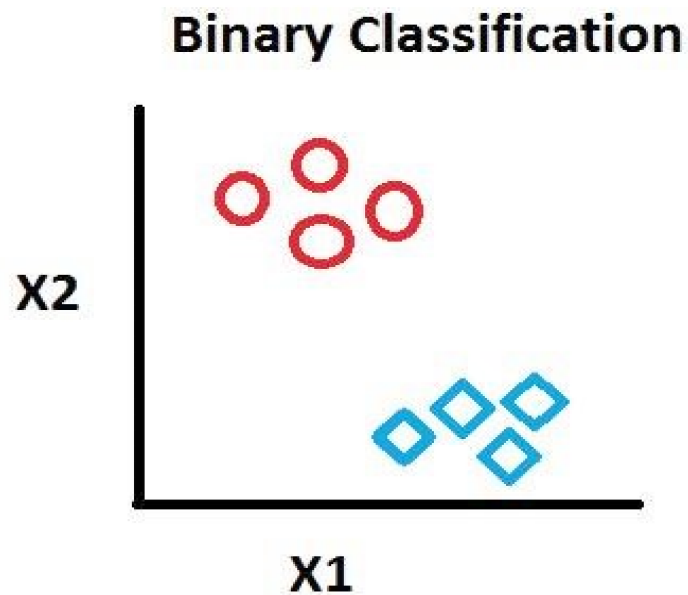
Đánh giá mô hình phân loại

- Hai vấn đề cơ bản của học máy:



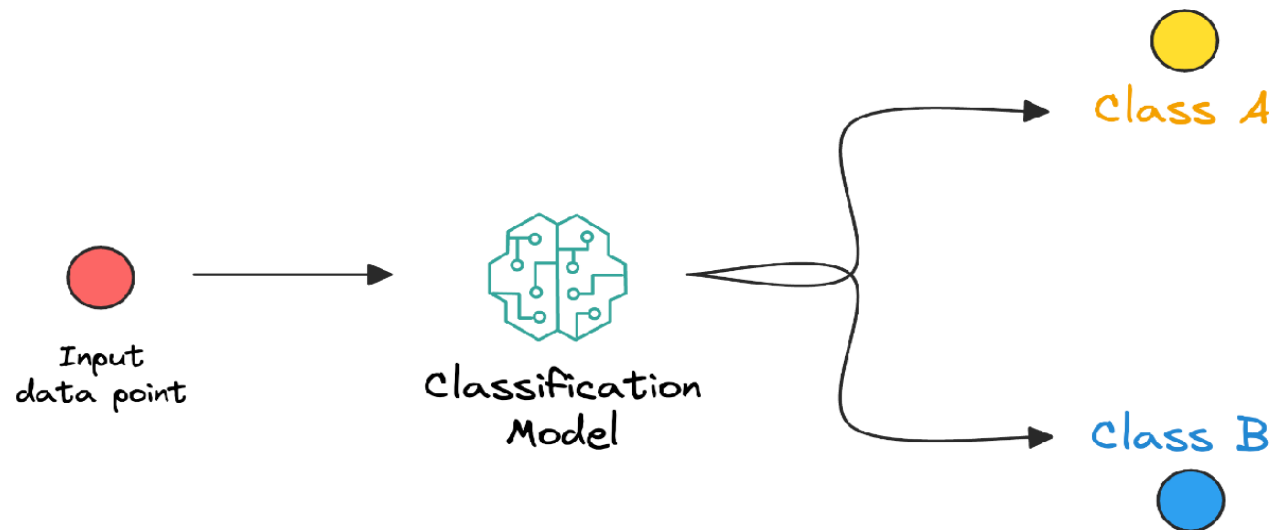
- Trong học máy, bài toán phân loại là dạng bài toán cơ bản dùng cho dự đoán dữ liệu đầu vào, mục tiêu là gán mỗi mẫu dữ liệu đầu vào vào một nhãn (class) xác định.
- Dữ liệu thường bao gồm các đặc trưng (features) và một nhãn mục tiêu (label) tương ứng.
- Mô hình học máy sẽ học từ dữ liệu huấn luyện để tìm ra ranh giới hoặc quy luật phân chia giữa các lớp.

- 2 dạng bài toán phân loại:
 - Phân loại nhị phân (binary classification).
 - Phân loại đa lớp (multi-class classification).



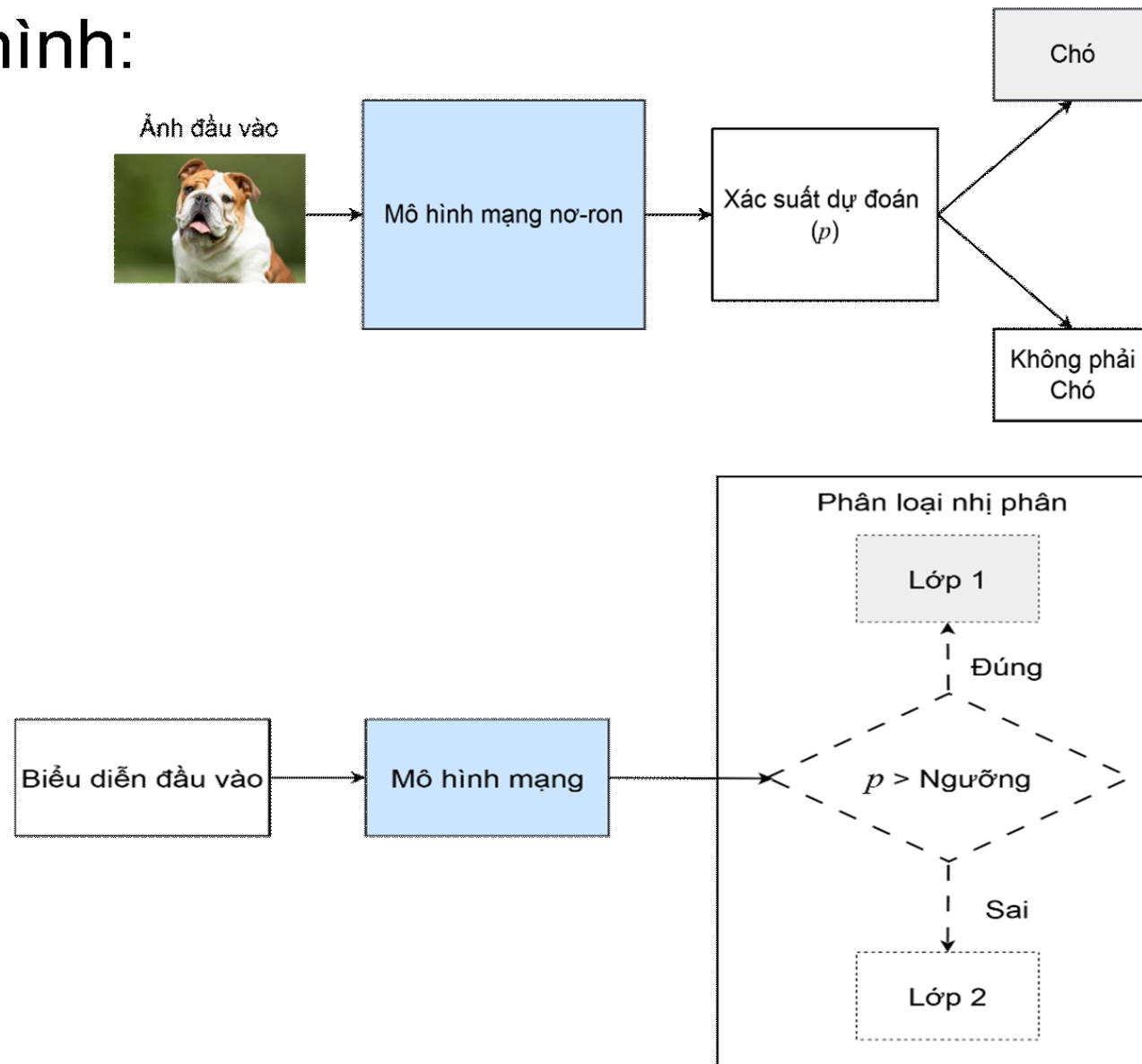
II. Phân loại nhị phân

- Phân loại nhị phân (binary classification) là một trong những vấn đề phổ biến.
- Mô hình mạng nơ-ron dự đoán một mẫu đầu vào thành một trong hai lớp (class).

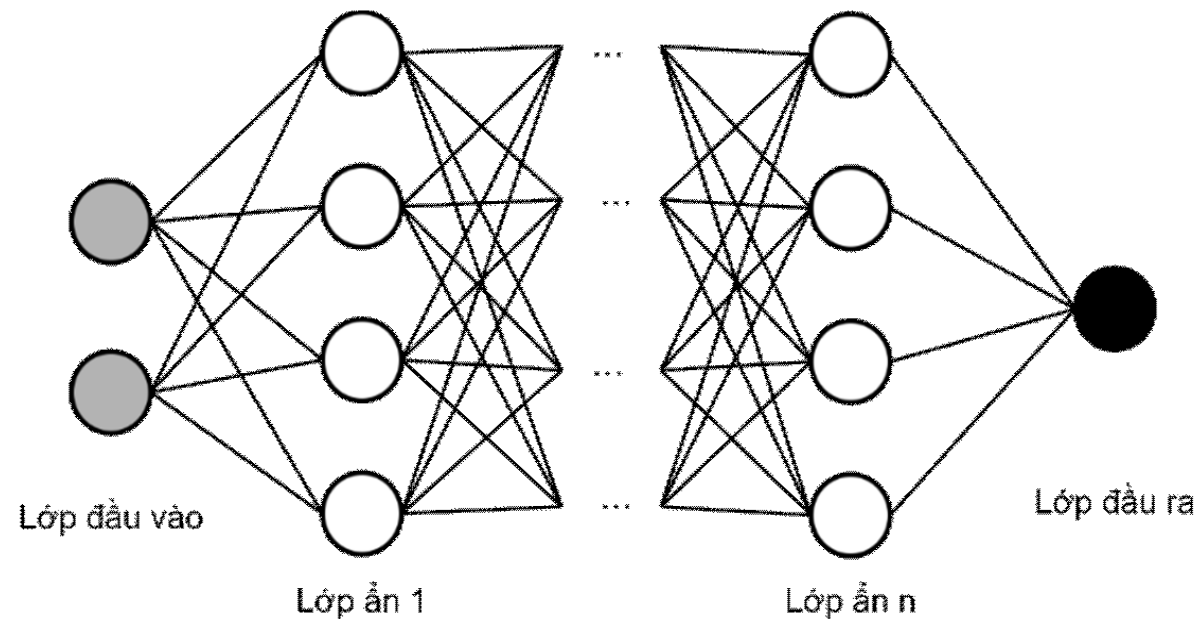


II. Phân loại nhị phân

■ Mô hình:



- Kiến trúc mạng:



- Hàm loss: Thường sử dụng hàm *Binary cross-entropy (BCE)*.

■ Ví dụ minh họa:

```
# BƯỚC 1: ĐỌC DỮ LIỆU TỪ FILE diabetes.csv VÀ CHIA DỮ LIỆU LÀM 2 PHẦN: TRAINING VÀ TESTING
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import keras

# 1.1. Đọc dữ liệu
data = pd.read_csv("Dataset/diabetes.csv")

# 1.2. Tách đặc trưng (X) và nhãn (y)
X = data.iloc[:, :-1] # tất cả cột trừ cột cuối
y = data.iloc[:, -1]  # cột cuối (label: có/không tiểu đường)

# 1.3. Chia train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 1.4. Chuẩn hóa dữ liệu
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
# X_test = scaler.transform(X_test)
y_train_onehot = keras.utils.to_categorical(y_train, 2)
y_test_onehot = keras.utils.to_categorical(y_test, 2)

# Hiển thị số bản ghi tập train/test
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

■ Ví dụ minh họa:

```
# BƯỚC 2: XÂY DỰNG MẠNG NƠ RON
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential
# Xây dựng mạng nơ-ron với 3 lớp ẩn cho bài toán phân loại nhị phân
def model_PhanLoaiNhiPhan():
    # Khởi tạo mạng
    model = Sequential()
    # Xây dựng mạng: 1 Input + 2 Hidden + 1 Output
    model.add(Dense(8, input_dim=6, activation='relu'))
    model.add(Dense(4, activation='relu'))

    model.add(Dense(2, activation='softmax'))

    # Return
    return model

# Chọn model (1 lớp ẩn hay nhiều lớp ẩn)
model = model_PhanLoaiNhiPhan()

# Hiển thị thông tin model (tùy chọn)
model.summary()

# Hàm Loss và optimizer
model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

■ Ví dụ minh họa:

```
# BƯỚC 4: HUẤN LUYỆN (TRAINING MODEL)
from tensorflow import keras
import matplotlib.pyplot as plt

# 4.1. Training model (có nhiều tùy chọn). Một số tham số:
history = model.fit(X_train, y_train_onehot, validation_split=0.2,
                    epochs=10, batch_size=16)

# 4.2. Lưu weight và model
model.save('modelNN_model.h5')

# BƯỚC 5: ĐÁNH GIÁ MODEL
```

■ Ví dụ minh họa:

```
# BƯỚC 6: SỬ DỤNG TRAINED MODEL DỰ ĐOÁN MỘT BỆNH NHÂN CỤ THỂ
from tensorflow.keras.models import Sequential, Model, load_model,
Sequential

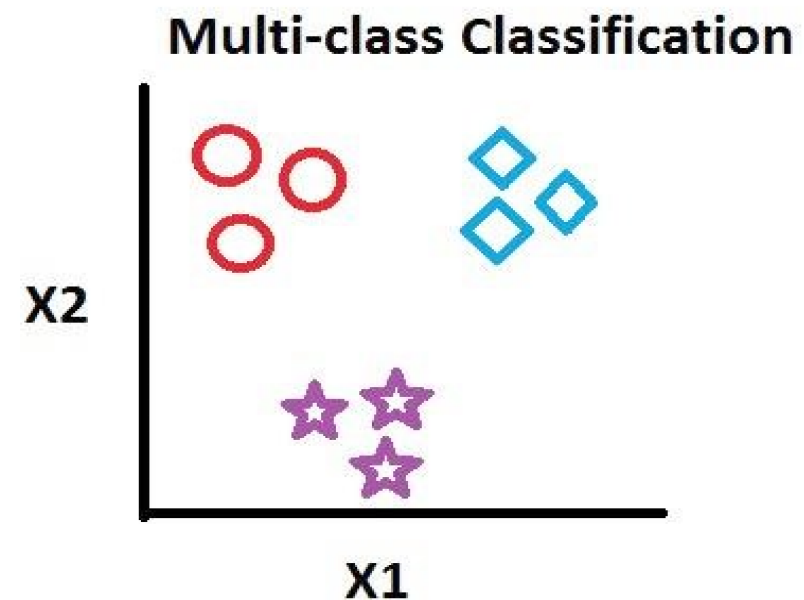
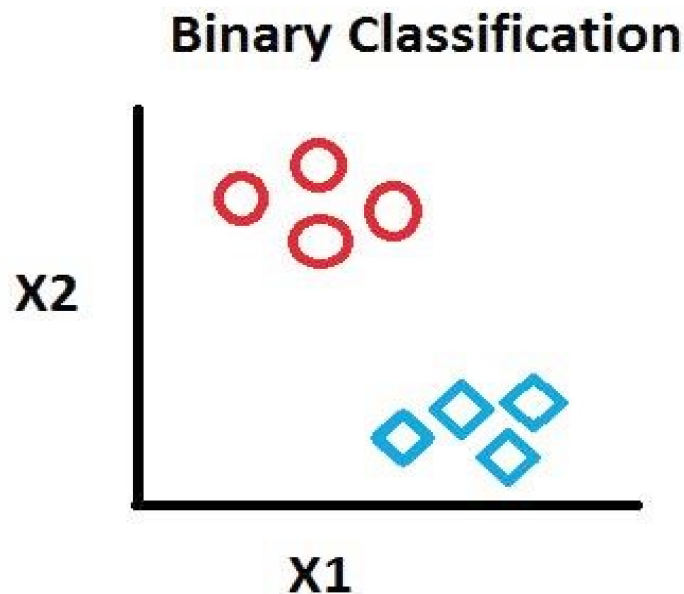
# 6.1. Load lại model
model_loaded = load_model('modelNN_model.h5')

# 6.2. Dự đoán 1 bệnh nhân cụ thể
features = [13.1976, 0, 50.0, 110.1, 10, 35.7, 10, 10]
results = model_loaded.predict([features])
print(results)
# results = np.argmax(results, axis=1)
# print("Dự đoán: ", results)

# 6.3. Xây dựng ứng dụng web (cần IDE như: Pycharm...)
```

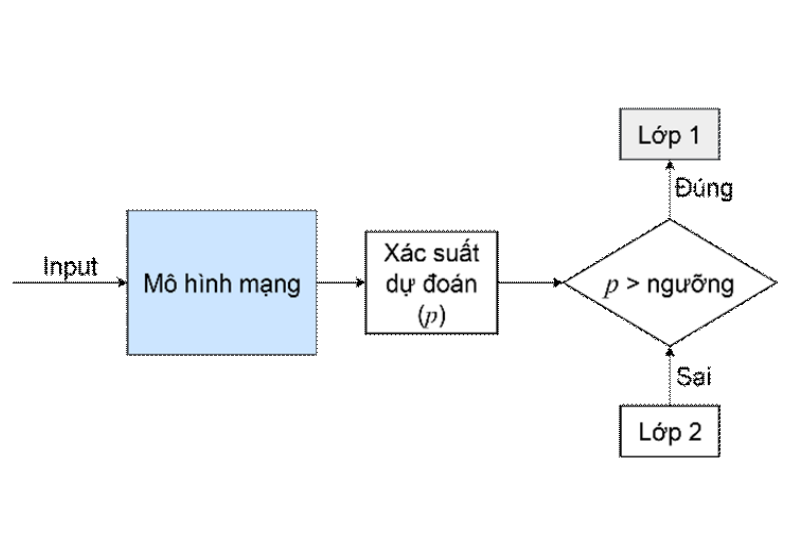
III. Phân loại đa lớp

- Trong học máy, phân loại đa lớp (hay phân loại đa danh mục - multi-class classification) đề cập đến vấn đề phân loại mẫu thành một trong nhiều lớp (từ ba trở lên).

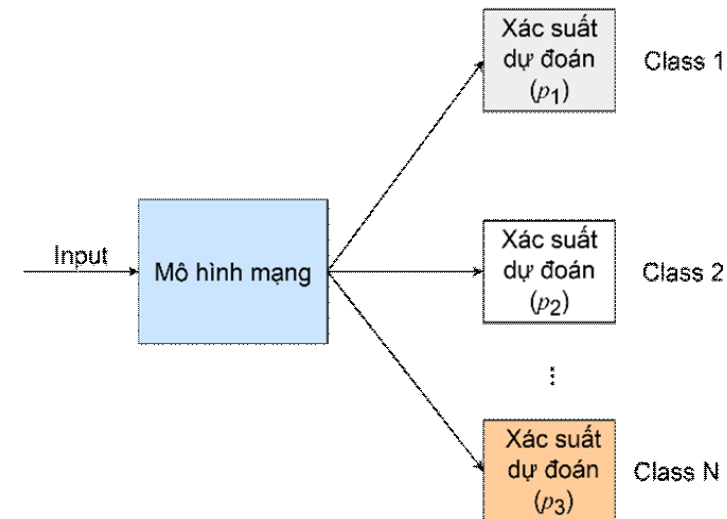


III. Phân loại đa lớp

- Các mô hình phân loại đa lớp tạo ra một xác suất cho mỗi lớp, trong đó tổng điểm xác suất đầu ra phải bằng 1.

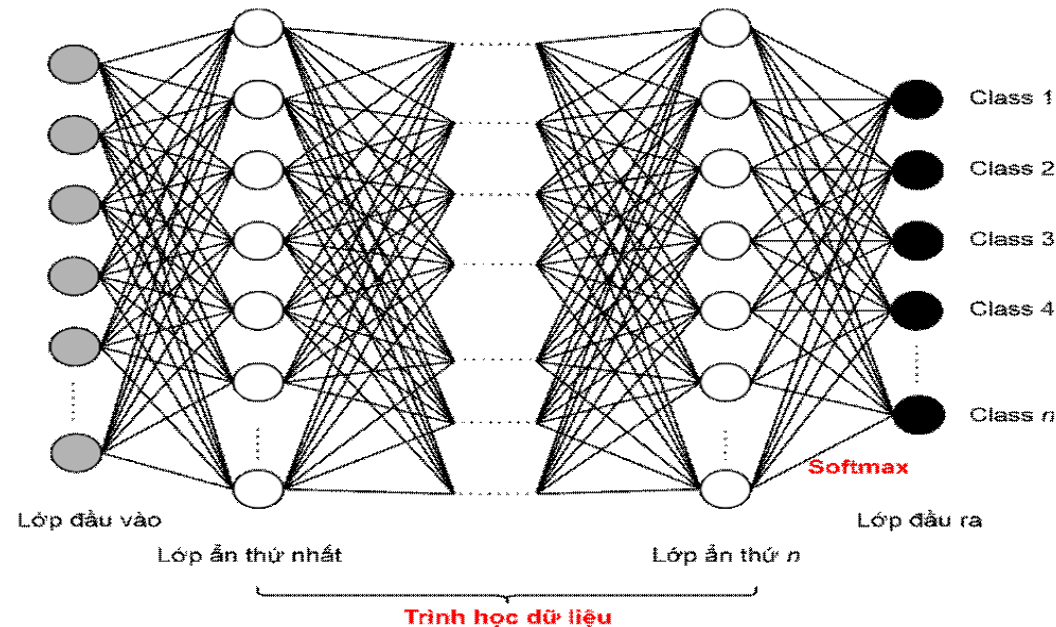


Phân loại nhị phân



Phân loại đa lớp

■ Kiến trúc mạng:



■ Hàm loss: Thường sử dụng hàm *categorical cross-entropy (CCE)*.

III. Phân loại đa lớp

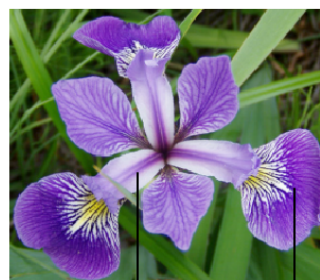
- Ví dụ minh họa: Xây dựng mô hình phân loại hoa iris (3 lớp).

iris setosa



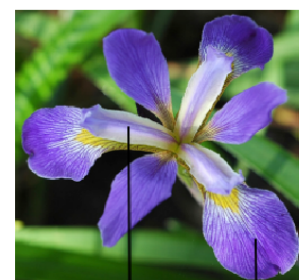
petal sepal

iris versicolor



petal sepal

iris virginica



petal sepal

	A	B	C	D	E	F
1	sepal length	sepal width	petal length	petal width	iris	iris_encoded
2	5.1	3.5	1.4	0.2	Iris-setosa	0
3	4.9	3	1.4	0.2	Iris-setosa	0
4	4.7	3.2	1.3	0.2	Iris-setosa	0
5	6.4	3.2	4.5	1.5	Iris-versicolor	1
6	6.9	3.1	4.9	1.5	Iris-versicolor	1
7	5.5	2.3	4	1.3	Iris-versicolor	1
8	7.1	3	5.9	2.1	Iris-virginica	2
9	6.3	2.9	5.6	1.8	Iris-virginica	2
10	6.5	3	5.8	2.2	Iris-virginica	2

■ Ví dụ minh họa:

```
# BƯỚC 1: ĐỌC DỮ LIỆU TỪ FILE IRIS.csv VÀ CHIA DỮ LIỆU LÀM 2 PHẦN: TRAINING VÀ TESTING
import pandas as pd
import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1.1. Đọc dữ liệu
data = pd.read_csv("Dataset/IRIS.csv")

# 1.2. Tách đặc trưng (X) và nhãn (y)
# Lấy cột 0,1,2,3 làm X và cột 5 làm Y
X = data.iloc[:, [0, 1, 2, 3]]
y = data.iloc[:, 5]

# 1.3. Chia train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 1.4. Chuẩn hóa dữ liệu
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
y_train_onehot = keras.utils.to_categorical(y_train, 3)
y_test_onehot = keras.utils.to_categorical(y_test, 3)

# Hiển thị số bản ghi tập train/test
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

■ Ví dụ minh họa:

```
# BƯỚC 2: XÂY DỰNG MẠNG NƠ RON
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential

# 2.1. Xây dựng mạng nơ-ron với 1 lớp ẩn
def IRIS_Network():
    # Khởi tạo mạng
    model = Sequential()

    # Xây dựng mạng
    model.add(Dense(8, input_dim=4, activation='relu'))

    model.add(Dense(3, activation='softmax')) # 3 hay số output classes

    # Return
    return model

# 2.2. Hiển thị thông tin model (tuỳ chọn)
model = IRIS_Network()
model.summary()

# Hàm Loss và optimizer
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

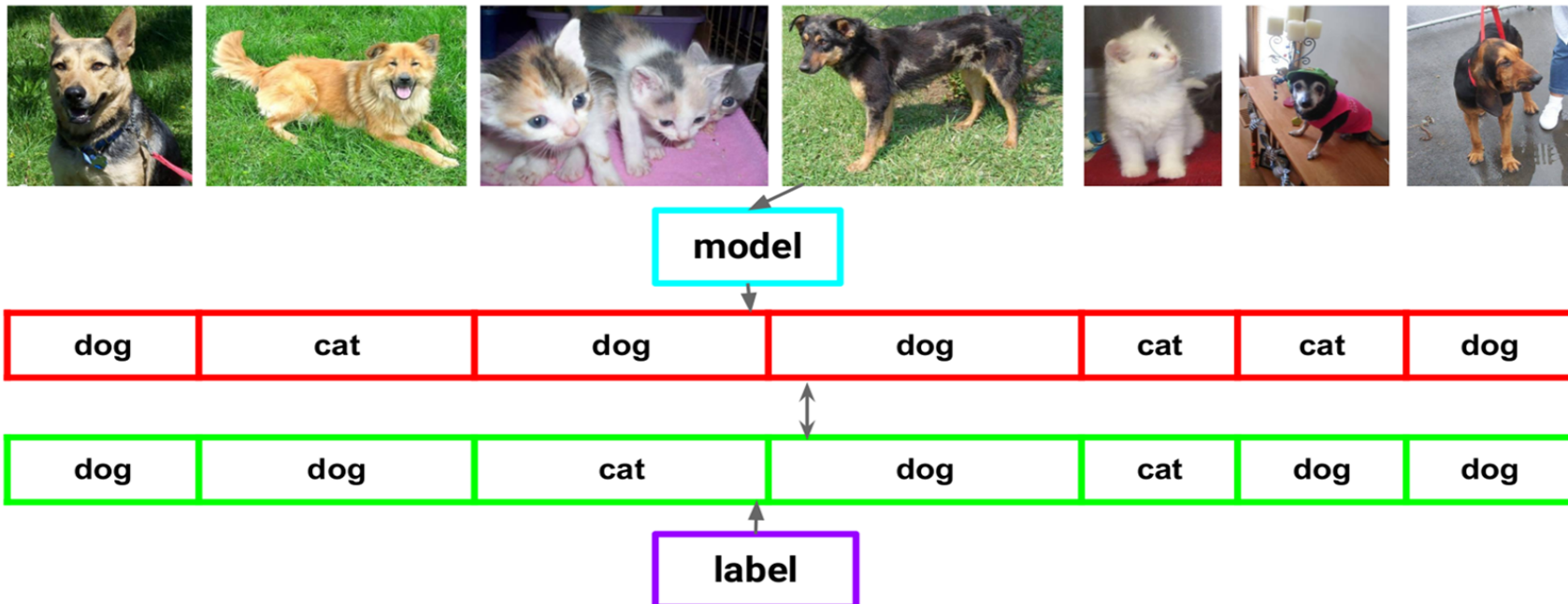
IV. Đánh giá mô hình phân loại

- Việc đánh giá hiệu quả mô hình là rất quan trọng để đảm bảo mô hình hoạt động đúng và phù hợp với bài toán cụ thể.
- Metric đánh giá cho bài toán phân loại:
 - Accuracy
 - Precision
 - Recall

- Metric đánh giá bài toán phân loại:
 - TP, FP, FN, TN.

Cách đánh giá mô hình trong Supervised Learning

Classification: dog vs cat



IV. Đánh giá mô hình phân loại

- Metric đánh giá bài toán phân loại:

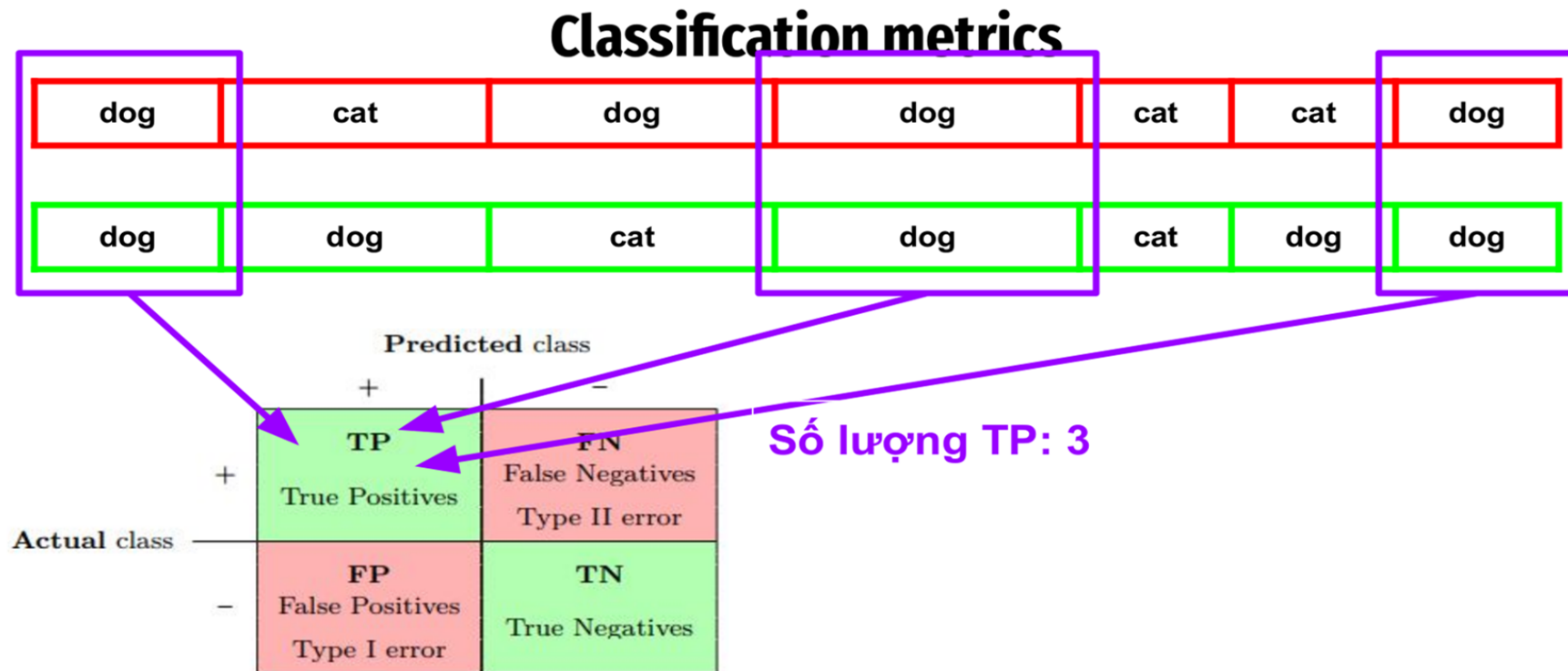
Classification metrics

dog	cat	dog	dog	cat	cat	dog
dog	dog	cat	dog	cat	dog	dog

		Predicted class	
		+	-
Actual class	+	TP True Positives	FN False Negatives Type II error
	-	FP False Positives Type I error	TN True Negatives

Class +: Class ta quan tâm: e.g. dog
Class -: class(es) còn lại: e.g. cat

- Metric đánh giá bài toán phân loại:



IV. Đánh giá mô hình phân loại

- Metric đánh giá bài toán phân loại:

Classification metrics

dog	cat	dog	dog	cat	cat	dog
dog	dog	cat	dog	cat	dog	dog

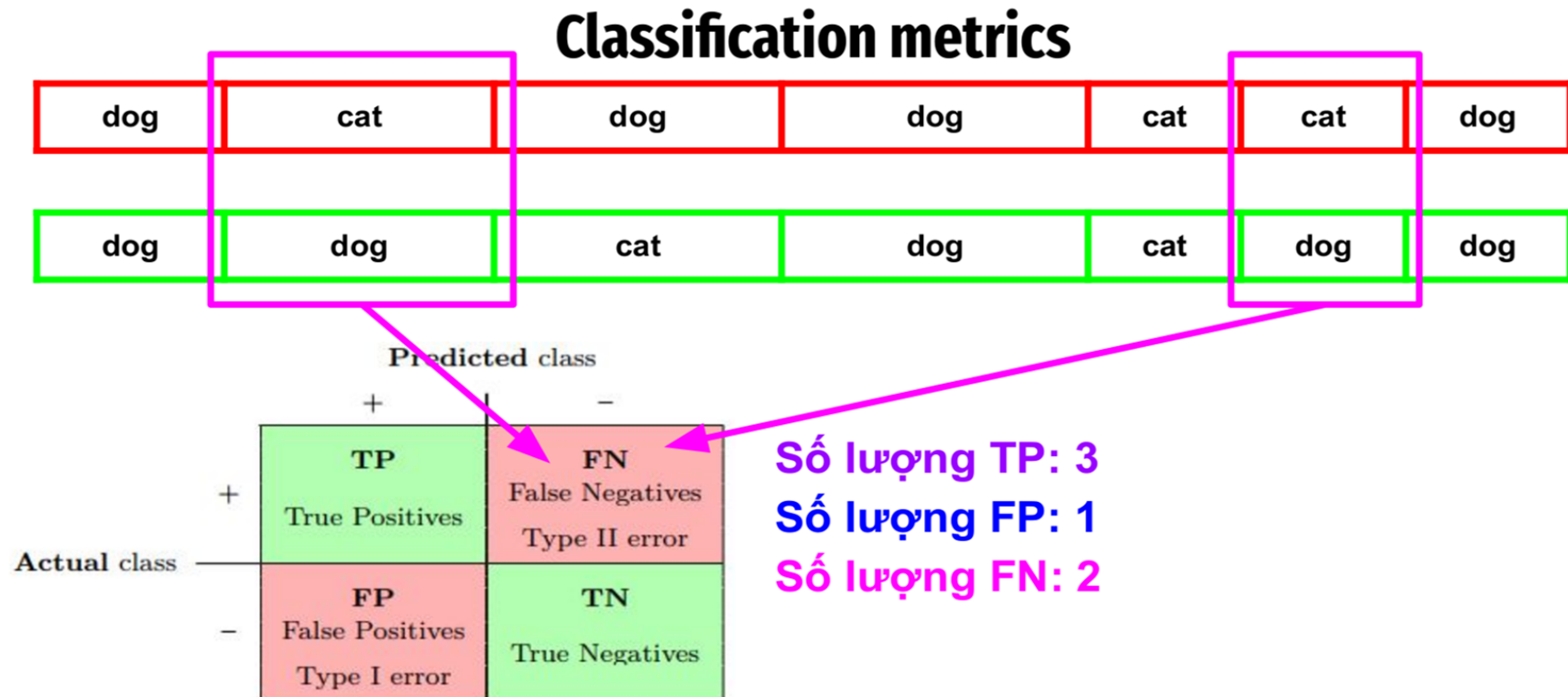
		Predicted class	
		+	-
Actual class	+	TP True Positives	FN False Negatives Type II error
	-	FP False Positives Type I error	TN True Negatives

Số lượng TP: 3

Số lượng FP: 1

IV. Đánh giá mô hình phân loại

- Metric đánh giá bài toán phân loại:



IV. Đánh giá mô hình phân loại

- Metric đánh giá bài toán phân loại:

Classification metrics

dog	cat	dog	dog	cat	cat	dog
dog	dog	cat	dog	cat	dog	dog

		Predicted class	
		+	-
Actual class	+	TP True Positives	FN False Negatives Type II error
	-	FP False Positives Type I error	TN True Negatives

Số lượng TP: 3

Số lượng FP: 1

Số lượng FN: 2

Số lượng TN: 1

IV. Đánh giá mô hình phân loại

- Metric đánh giá bài toán phân loại:

Metric	Công thức	Ý nghĩa
Accuracy	$(TP+TN)/All$	Độ chính xác tổng quát
Precision	$TP/(TP+FP)$	Độ chính xác đối với class +
Recall	$TP/(TP+FN)$	Độ bao phủ đối với các dự đoán về class +
F1 score	$(2*Pre*Re)/(Pre+Re)$: Trung bình điều hòa giữa Precision và Recall	

		Predicted class	
		+	-
Actual class	+	TP True Positives	FN False Negatives Type II error
	-	FP False Positives Type I error	TN True Negatives

Số lượng TP: 3 Accuracy = $(3+1)/7$

Số lượng FP: 1 Precision = $3/(3+1)$

Số lượng FN: 2 Recall = $3/(3+2)$

Số lượng TN: 1 F1 score =

All: 7 $(2*0.75*0.6)/(0.75+0.6)$

■ Ví dụ minh họa:

```
# BƯỚC 1: ĐỌC DỮ LIỆU TỪ FILE IRIS.csv VÀ CHIA DỮ LIỆU LÀM 2 PHẦN: TRAINING VÀ TESTING
import pandas as pd
import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1.1. Đọc dữ liệu
data = pd.read_csv("Dataset/IRIS.csv")

# 1.2. Tách đặc trưng (X) và nhãn (y)
# Lấy cột 0,1,2,3 làm X và cột 5 làm Y
X = data.iloc[:, [0, 1, 2, 3]]
y = data.iloc[:, 5]

# 1.3. Chia train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 1.4. Chuẩn hóa dữ liệu
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
y_train_onehot = keras.utils.to_categorical(y_train, 3)
y_test_onehot = keras.utils.to_categorical(y_test, 3)

# Hiển thị số bản ghi tập train/test
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

■ Ví dụ minh họa:

```
# BƯỚC 2: XÂY DỰNG MẠNG NƠ RON
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential

# 2.1. Xây dựng mạng nơ-ron với 1 lớp ẩn
def IRIS_Network():
    # Khởi tạo mạng
    model = Sequential()

    # Xây dựng mạng
    model.add(Dense(8, input_dim=4, activation='relu'))

    model.add(Dense(3, activation='softmax')) # 3 hay số output classes

    # Return
    return model

# 2.2. Hiển thị thông tin model (tuỳ chọn)
model = IRIS_Network()
model.summary()

# BƯỚC 3: LỰA CHỌN HÀM MẤT MẮT VÀ TỐI ƯU HOÁ
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

■ Ví dụ minh họa:

```
# BƯỚC 4: HUẤN LUYỆN (TRAINING MODEL)
from tensorflow import keras
import matplotlib.pyplot as plt

# 4.1. Training model (có nhiều tùy chọn). Một số tham số:
history = model.fit(X_train, y_train_onehot, validation_split=0.2,
                    epochs=10, batch_size=16)

# 4.2. Lưu weight và model
model.save('modelNN_model.h5')

# 4.3. Hiển thị Training and Validation Loss
plt.figure(figsize=(8, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

■ Ví dụ minh họa:

```
# BƯỚC 5: ĐÁNH GIÁ (TESTING) MÔ HÌNH
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

# 5.1. Hiển thị kết quả (accuracy) trên tập testing
test_loss, test_acc = model.evaluate(X_test, y_test_onehot)
print('Test loss:', test_loss)
print('Test accuracy:', test_acc)

# 5.2. Hiển thị các kết quả khác (Accuracy, Precision, Recall, F1 score)
trên tập testing.
y_test_predicted = model.predict(X_test)
y_test_predicted=np.argmax(y_test_predicted, axis=1)

print('Accuracy:', accuracy_score(y_test,y_test_predicted))
print('Precision:', precision_score(y_test,y_test_predicted))
print('Recall:', recall_score(y_test, y_test_predicted))
print('F1 score:', f1_score(y_test, y_test_predicted))
```

- [1]. <https://medium.com/>
- [2]. <https://towardsdatascience.com/>
- [3]. <https://machinelearningcoban.com/>
- [4]. Huang Shih-Chia, and Trung-Hieu Le. *Principles and labs for deep learning*. Academic Press, 2021.

Thank You !

Question?